1. Práctica 7: Conexión con periféricos externos - Display y drivers

1.1. Objetivos

- Realizar conexión con periféricos externos, en concreto el display.
- Programar un driver sencillo para controlar el pintado.

1.2. Fundamentos teóricos

Cualquier procesador, sea del tipo que sea, se acompaña siempre, además de los periféricos internos, de periféricos externos. De lo contrario, la interfaz de comunicación con el usuario resultaría imposible. Desde simples pines con los que comunicar vía `UART`, como ya hemos visto, hasta sistemas complejos como los que veremos en esta práctica. Las conexiones de las que dispone nuestra placa **32F723EDISCOVERY**se pueden observar en `um2140 5`, aunque las reproducimos en la Figura 1 para facilitar la consulta.

En concreto, observando dicha imagen, vemos que el display (a la izquierda) se conecta con la placa a través de GPIOs, y un periférico interno conocido como `FMC`. El `FMC` o *Flexible Memory Controller* (`rm0431 12`) es un periférico que permite gestionar la conexión con memorias externas de diferentes tipos (SDRAM, NAND flash, PSRAM, NOR flash...) además de dispositivos basados en memorias, como los displays (que guardan la imagen de manera similar a como una memoria guarda datos). Esto libera la memoria interna del **STM32F723IE**para poder guardar programas, en lugar de pesadas imágenes.

Dichas conexiones requieren el manejo simultáneo de múltiples pines de entrada y salida, lo cual sería imposible de realizar directamente con el procesador. Por tanto, este periférico `FMC` permite, con una configuración apropiada, gestionar las transferencias de datos de manera autónoma. Podemos efectivamente comprobar que el display está conectado al `FMC` observando `um2140 5.14`. Adicionalmente, en `um2140 6.7`, nos dice qué pines se utilizan para las conexiones. (Estos nombres no son directamente los puertos GPIO de la placa, que se pueden consultar en `um2140 Table 19`.) Cruzando estos datos con `rm0431 12.3`, podemos ver qué señales concretas se utilizan para esta placa concreta.

Pero hay una peculiaridad, y es que el `FMC` en este caso no se comunica directamente con el display, sino que, de por medio, hay un chip controlador `ST7789H2`. Este chip está encargado de recibir comandos a través del `FMC`, configurando la memoria del display, así como sus características. Así pues, toda la información o comandos que queramos pasar al display deberán pasar por el `FMC`.

Adicionalmente, el propio panel LCD (modelo `FRD154BP2901` de *Frida*) cuenta con unas conexiones especiales que no controla el `ST7789H2`. Estas conexiones, concretamente, son tres pines (`um2140 Table 14`): PH7 (resetea el panel), PH11 (se encarga de encender la luz trasera) y PC8 (Interrupción de Tear Effect, no hace falta utilizar este último).

Es decir, que parte del control lo haremos directamente, y parte a través del periférico `FMC`. Podemos ver todas estas conexiones de manera simplificada en Figura 2.

Ahora, con todas las conexiones hechas, hay que activar el panel. Para activar periféricos, se siguen ciertas secuencias de inicialización que ponen el hardware en un estado inicial. Aunque

Figure 3. Hardware block diagram

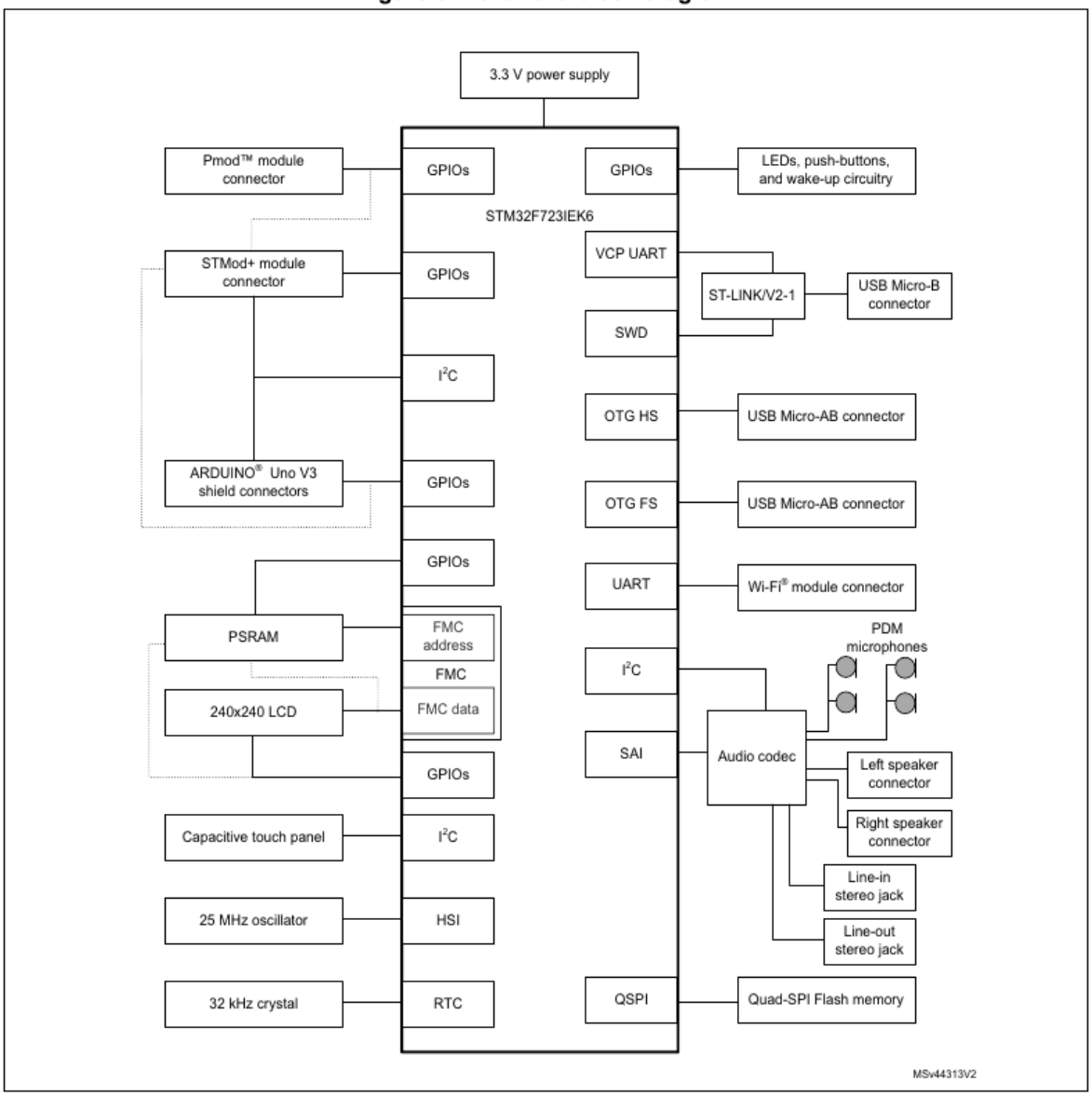


Figura 1: Diagrama de bloques de la placa **32F723EDISCOVERY**

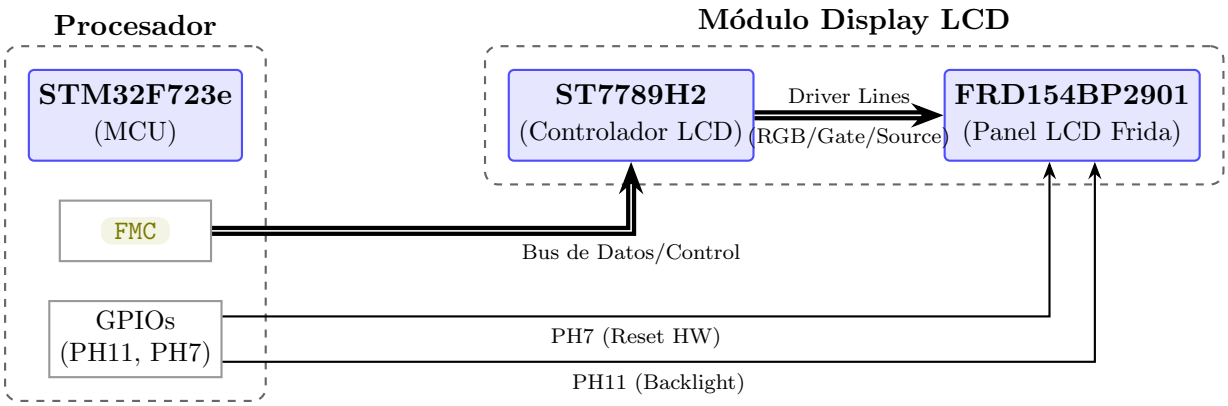


Figura 2: Diagrama de conexiones entre la MCU STM32F723, el controlador ST7789H2 y el panel LCD.

la documentación no es muy clara, si os fiáis de vuestro profesor, y no habéis abandonado la asignatura por dificultad (espero que no, si es así contactadme), el proceso no es excesivamente complicado. Habrá primero que activar el panel a través de los pines de conexión directa (básicamente, resetearlo), y luego activar el controlador `ST7789H2` utilizando comandos enviados por el `FMC`. En concreto, la secuencia necesaria de encendido es:

1. Encender la luz trasera del display desde el pin PH11 (esto es opcional, pero querremos ver la pantalla, no?).
2. Aplicar la secuencia de reset en `PH7`: `0` (esperar 5 ms) → `1` (esperar 10 ms) → `0` (esperar 20 ms) → `1` (esperar 10 ms).
3. Con el panel encendido, toca el turno al controlador, al que tenemos que mandar un comando de `SWRESET`.
   a) Mandar un comando de `SLEEP_IN` `ST7789H2 9.1.11` para asegurar que el panel está desactivado. La documentación obliga a esperar 5ms, esperamos 10ms para asegurar.
   b) Mandar un comando de `SWRESET` `ST7789H2 9.1.2` para resetear el panel. La documentación obliga a esperar 120ms, esperamos 200ms para asegurar.
   c) Mandar un comando de `SLEEP_OUT` `ST7789H2 9.1.12` para activar el panel. La documentación obliga a esperar 5ms, pero como menciona 120ms también, esperamos ese tiempo para asegurar.

Lo más interesante aquí es que estos comandos que debe recibir el panel LCD, se van a enviar a través del `FMC`. Es decir, que estamos delegando la configuración en este periférico versátil, en lugar de tener que hacerlo nosotros a través de los aproximadamente 30 pines que lo controlan. Y ahora, cómo funciona todo esto?

1.2.1. Funcionamiento del periférico FMC

El periférico `FMC` funciona, a ojos del programador, como una extensión de la memoria RAM del sistema. En lugar de requerir un protocolo de comunicación complejo con llamadas a funciones de transmisión bit a bit, el `FMC` mapea el controlador externo directamente en el mapa de memoria del microcontrolador (`rm0431 12.4`).

Esto significa que escribir o leer en el controlador de pantalla es tan sencillo como hacer una operación de escritura en una dirección de memoria en C. El propio hardware del `FMC` se encarga de traducir esa escritura en las señales eléctricas apropiadas (bus de datos, activación de `CS`, pulsos de `WR/RD`, etc.) a través de los pines GPIO asignados. En este caso, únicamente se mapean dos direcciones: `REG` (para indicar el registro o comando del periférico) y `RAM` (para indicar el valor que se escribe):

```
typedef struct
{
    volatile uint16_t REG;
    volatile uint16_t RAM;
} FMC_CONTROLLER_TypeDef;

#define FMC_BANK2           ((FMC_CONTROLLER_TypeDef *) FMC_BANK2_BASE_ADDR)

static inline void FMC_BANK2_WriteReg(uint8_t Reg)
{
```

```
    /* Write 16-bit Index, then write register */
    FMC_BANK2->REG = Reg;
    __DSB();
}

static inline void FMC_BANK2_WriteData(uint16_t Data)
{
    /* Write 16-bit Reg */
    FMC_BANK2->RAM = Data;
    __DSB();
}
```

Para comunicarse con el ST7789H2, el FMC utiliza una interfaz de tipo 8080/6800 paralela de tipo SRAM. Ésta contiene líneas que doblan para datos o comandos, y su información se discierne mediante una línea especial A0 (o Data/Command Line) (um2140 Table 14):

- Cuando la línea A0 está a nivel bajo (0), la transferencia se interpreta como un comando o instrucción para el controlador.
- Cuando la línea A0 está a nivel alto (1), la transferencia se interpreta como datos (parámetros del comando o colores de los píxeles).

Gracias al mapeo en memoria, en este caso en el banco 2 (BANK2) del FMC, diferenciar entre enviar un comando o un dato es tan simple como escribir en dos direcciones de memoria consecutivas dentro del rango asignado al FMC. La dirección base (REG) enviará la línea A0 a 0 (Comandos), mientras que la siguiente (RAM) enviará A0 a 1 (Datos).

1.2.2. Funcionamiento del controlador ST7789H2

El chip ST7789H2 es el "cerebro" interno del panel LCD. Su función principal es mantener la imagen visible en la pantalla refrescando continuamente los píxeles físicos, liberando por completo al **STM32F723IE**.

Para lograr esto, el ST7789H2 contiene una memoria RAM interna de 240×320 localizaciones de memoria, que se configura mediante secuencias de comandos y datos concretas a través del FMC. El flujo general de trabajo con este chip consta de tres etapas:

1. **Configuración e Inicialización:** Tras la secuencia de reset, se le envían comandos mediante el FMC para definir el formato de color (por ejemplo, RGB565 de 16 bits por píxel), orientación de la pantalla (lectura vertical u horizontal), etc.
2. **Definición de Ventana de Trabajo:** Antes de pintar, se envía un comando de dirección de columnas (CASET, ST7789H2 9.1.20) y de filas (RASET, ST7789H2 9.1.21). Esto le dice al ST7789H2 en qué sub-región o rectángulo de la pantalla vamos a escribir.
3. **Escritura de Píxeles:** Una vez delimitada la zona, se envía el comando de escritura en memoria (RAMWR, ST7789H2 9.1.22). A partir de ese momento, cada dato enviado por el FMC corresponderá al color de un píxel (se pueden enviar varios en ráfaga). El propio controlador incrementa automáticamente la dirección de memoria interna píxel a píxel, llenando la ventana definida de izquierda a derecha y de arriba a abajo.

Es decir, el procesador escribe los colores a cada píxel en la memoria mediante el `FMC`, el `ST7789H2` los almacena en su RAM interna y sus circuitos de control (drivers (no confundir con driver software)) convierten esos valores digitales en las tensiones físicas que configuran el display `FRD154BP2901`. Un diagrama mostrando este funcionamiento se observa en la Figura 3.

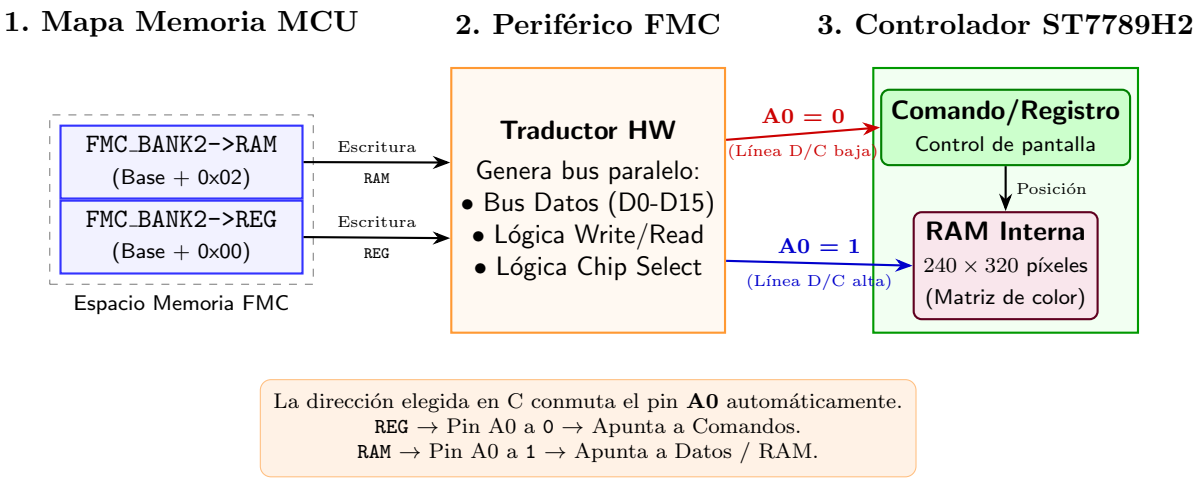


Figura 3: Mapeo de memoria a través del FMC hacia la lógica y RAM interna del ST7789H2

1.3. Desarrollo de la práctica

Para esta práctica, vamos a partir del código final de `main.c` al que queremos llegar. Básicamente, inicializaremos todos los elementos necesarios (el panel LCD, el periférico `FMC`, y el controlador `ST7789H2`), antes de dibujar algo sencillo en la pantalla. Se incluye además un pequeño código para poner píxeles aleatorios y ver que de verdad está funcionando:

```c
// Definimos una variable global para la semilla (debe ser distinta de 0)
static uint32_t estado_rng = 123456789;

// Genera un entero sin signo de 32 bits (0 a 4,294,967,295)
uint32_t rand32(void) {
    uint32_t x = estado_rng;
    x ^= x << 13;
    x ^= x >> 17;
    x ^= x << 5;
    return estado_rng = x;
}

int main(void)
{
    LCD_Init();
    FSMC_Init();
    ST7789H2_INIT();

    /* Clear the LCD */
    for (int i = 0; i < 240; i++)
        for (int j = 0; j < 240; j++)
            ST7789H2_WritePixel(i, j, LCD_COLOR_WHITE);


    for (int i = 50; i < 60; i++)
        for (int j = 50; j < 60; j++)
            ST7789H2_WritePixel(i, j, LCD_COLOR_RED);

```

```
29
30      while(1)
31      {
32          uint32_t rnd = rand32();
33          ST7789H2_WritePixel((rnd >> 24) & 0xff, (rnd >> 16) & 0xff, rnd & 0xffff
                );
34      }
35  }
```

Yendo por orden, lo primero es inicializar el panel LCD. Para ello hay que configurar los pines de conexión directa (de reset, interrupción (que no utilizaremos) y luz trasera). Posteriormente, aplicaremos la secuencia de inicialización del panel.

```
1   void LCD_Pins_Init() {
2       /* Initialize LCD special pins GPIOs */
3       /* LCD_RESET GPIO configuration */
4       RCC_AHB1ENR->bits.GPIOHEN = 1;
5       GPIO_CONFIG(GPIOH, 7, GPIO_MODE_OUTPUT, GPIO_OTYPE_PP, GPIO_OSPEED_HIGH,
            GPIO_PUPD_NONE, 0);
6       /* LCD_TE GPIO configuration */
7       RCC_AHB1ENR->bits.GPIOCEN = 1;
8       GPIO_CONFIG(GPIOC, 8, GPIO_MODE_INPUT, GPIO_OTYPE_PP, GPIO_OSPEED_HIGH,
            GPIO_PUPD_NONE, 0);
9       /* LCD_BL_CTRL GPIO configuration */
10      RCC_AHB1ENR->bits.GPIOHEN = 1;
11      GPIO_CONFIG(GPIOH, 11, GPIO_MODE_OUTPUT, GPIO_OTYPE_PP, GPIO_OSPEED_LOW,
            GPIO_PUPD_NONE, 0);
12  }
13
14  void LCD_Startup_Seq() {
15      /* Backlight enable */
16      GPIO_PIN_WRITE(GPIOH, 11, GPIO_STATE_ONE);
17
18      /* Apply hardware  */
19      //... completar escribir en el pin de reset la secuencia de activación
20      //... utiliza la función DELAY para las esperas
21  }
22
23  void LCD_Init() {
24      LCD_Pins_Init();
25      LCD_Startup_Seq();
26  }
```

Una vez iniciado el panel, lo siguiente es activar el `FMC`. La configuración es bastante complicada y ya está hecha en el código proporcionado de base. Se recomienda echar un vistazo. En resumen, lo que hacemos es activar primero todos los pines requeridos por el controlador, y luego configurar el modo de conexión con la memoria, así como las características de temporización del bus.

Finalmente, se configura el controlador del display, enviando comandos directamente desde el `FMC`. El código se encuentra en `ST7789h2.h`, y hay que completar algunas partes para hacerlo funcionar completamente. En concreto:

En la función `ST7789H2_INIT`, al principio, debemos inicializar el controlador, mandando los comandos adecuados:

```
1   /* Sleep In Command */
2   ST7789H2_WriteReg(ST7789H2_SLEEP_IN, (uint8_t*)NULL, 0);
3   /* Wait for 10ms */
4   DELAY(10);
```

```
5
6 //... completar mandar el comando de RESET, seguido de una pausa de 200ms
7 //... completar mandar el comando de SLEEP_OUT, seguido de una pausa de 120ms
```

También deberemos crear la función `DISPLAY_ON` para encender el display:

```
1 void ST7789H2_DisplayOn(void)
2 {
3     //... completar mandar el comando DISPLAY_ON
4     //... completar mandar el comando SLEEP_OUT
5 }
```

Con esto ya deberíamos tener el display funcionando! El código debería borrar el display a blanco (tardará un poco), pintar un cuadrado rojo, y luego empezar a dibujar cuadrados de colores aleatoriamente.

1.3.1. Creación de la Librería Gráfica (GFX Library)

Dibujar gráficos solo con `ST7789H2_WritePixel()` resulta tedioso. Para abstraer el dibujado, se diseñan librerías gráficas que faciliten el proceso.

Esta librería (implementada por ejemplo en dos archivos llamados `gfx.h` y `gfx.c`) debe apoyarse directamente sobre la función de bajo nivel `ST7789H2_WritePixel(x, y, color)` proporcionada por el driver del display. A continuación se proporciona un **ejemplo** de qué funciones podrían implementarse, si bien puedes elegir las que más te convengan.

Fichero de cabecera (`gfx.h`): Crea el fichero `gfx.h` con las definiciones necesarias y los prototipos de las funciones:

```
1  #ifndef GFX_H_
2  #define GFX_H_
3
4  #include <stdint.h>
5  #include <stdlib.h>
6  #include "ST7789h2.h"
7
8  #define GFX_WIDTH   240
9  #define GFX_HEIGHT  240
10
11 /* Primitivas basicas con verificacion de limites */
12 void GFX_DrawPixel(int16_t x, int16_t y, uint16_t color);
13 void GFX_DrawFastHLine(int16_t x, int16_t y, int16_t w, uint16_t color);
14 void GFX_DrawFastVLine(int16_t x, int16_t y, int16_t h, uint16_t color);
15
16 /* Lineas genericas */
17 void GFX_DrawLine(int16_t x0, int16_t y0, int16_t x1, int16_t y1, uint16_t color
       );
18
19 /* Cuadrados y Rectangulos */
20 void GFX_DrawRect(int16_t x, int16_t y, int16_t w, int16_t h, uint16_t color);
21 void GFX_FillRect(int16_t x, int16_t y, int16_t w, int16_t h, uint16_t color);
22 void GFX_DrawSquare(int16_t x, int16_t y, int16_t size, uint16_t color);
23 void GFX_FillSquare(int16_t x, int16_t y, int16_t size, uint16_t color);
24
25 /* Circulos */
26 void GFX_DrawCircle(int16_t x0, int16_t y0, int16_t r, uint16_t color);
```

```
void GFX_FillCircle(int16_t x0, int16_t y0, int16_t r, uint16_t color);

/* Control de pantalla */
void GFX_FillScreen(uint16_t color);

#endif /* GFX_H_ */
```

Implementación (`gfx.c`): A continuación se describen los requisitos algorítmicos que debes codificar dentro de `gfx.c`:

1. **Trazado rápido y comprobación de límites (`GFX_DrawPixel`, `GFX_DrawFastHLine`, `GFX_DrawFastVLine`):** Implementa `GFX_DrawPixel` asegurando que no se dibujen coordenadas fuera del rango de la pantalla. Igualmente implementa funciones rápidas (un solo bucle) para dibujar líneas horizontales y verticales.

2. **Algoritmo de Línea de Bresenham (`GFX_DrawLine`):** Para dibujar líneas arbitrarias entre (x_0, y_0) y (x_1, y_1), debes implementar el algoritmo de *Bresenham*. Este algoritmo utiliza únicamente aritmética entera para calcular los píxeles intermedios:

```
void GFX_DrawLine(int16_t x0, int16_t y0, int16_t x1, int16_t y1, uint16_t
    color) {
    int16_t dx = abs(x1 - x0);
    int16_t dy = abs(y1 - y0);
    int16_t sx = (x0 < x1) ? 1 : -1;
    int16_t sy = (y0 < y1) ? 1 : -1;
    int16_t err = dx - dy;

    while (1) {
        GFX_DrawPixel(x0, y0, color);
        if (x0 == x1 && y0 == y1) break;
        int16_t e2 = 2 * err;
        if (e2 > -dy) { err -= dy; x0 += sx; }
        if (e2 < dx)  { err += dx; y0 += sy; }
    }
}
```

3. **Dibujado de Rectángulos y Cuadrados:**
   - `GFX_DrawRect`: Dibuja solo el borde de un rectángulo mediante líneas horizontales y verticales. Reaprovecha las funciones anteriores.
   - `GFX_FillRect`: Rellena el área interna dibujando múltiples líneas horizontales.
   - `GFX_DrawSquare` y `GFX_FillSquare`: Reutilizan las funciones para un rectángulo con lados iguales.

4. **Algoritmo del Punto Medio para Círculos (`GFX_DrawCircle` y `GFX_FillCircle`):** Para dibujar el contorno de un círculo de radio r centrado en un punto (x_0, y_0), existe el denominado *algoritmo del punto medio de Bresenham* (este tío debió hacer muchas cosas por los gráficos 2D sencillos). La idea detrás del mismo es dibujar solo un arco de 45 grados, y utilizar la simetría en octantes del círculo (básicamente, un círculo se forma de 8 partes iguales, como una pizza). Para la versión rellena (`GFX_FillCircle`), sustituye los puntos individuales por líneas verticales que unan los límites simétricos superiores e inferiores:

```
void GFX_DrawCircle(int16_t x0, int16_t y0, int16_t r, uint16_t color) {
    int16_t f = 1 - r;
    int16_t ddF_x = 1;
```

```
    int16_t ddF_y = -2 * r;
    int16_t x = 0;
    int16_t y = r;

    GFX_DrawPixel(x0, y0 + r, color);
    GFX_DrawPixel(x0, y0 - r, color);
    GFX_DrawPixel(x0 + r, y0, color);
    GFX_DrawPixel(x0 - r, y0, color);

    while (x < y) {
        if (f >= 0) {
            y--;
            ddF_y += 2;
            f += ddF_y;
        }
        x++;
        ddF_x += 2;
        f += ddF_x;

        GFX_DrawPixel(x0 + x, y0 + y, color);
        GFX_DrawPixel(x0 - x, y0 + y, color);
        GFX_DrawPixel(x0 + x, y0 - y, color);
        GFX_DrawPixel(x0 - x, y0 - y, color);
        GFX_DrawPixel(x0 + y, y0 + x, color);
        GFX_DrawPixel(x0 - y, y0 + x, color);
        GFX_DrawPixel(x0 + y, y0 - x, color);
        GFX_DrawPixel(x0 - y, y0 - x, color);
    }
}
```

Comprobación en `main.c`: Añade pruebas en `main.c`, incluyendo tu librería `"gfx.h"` y ejecutando un test completo que dibuje líneas diagonales, rectángulos, cuadrados, círculos... de distintos colores.

1.4. Para hacer más

Explora la documentación del display para estudiar las maneras más rápidas de pintar la pantalla (por líneas en lugar de píxeles, por ejemplo). También puedes aumentar la velocidad del procesador cambiando su frecuencia de reloj de los 16MHz por defecto a más de 200MHz para conseguir mayor velocidad.